

Aula 11: Quicksort

Ordenação rápida com dividir para conquistar

Onde chegamos

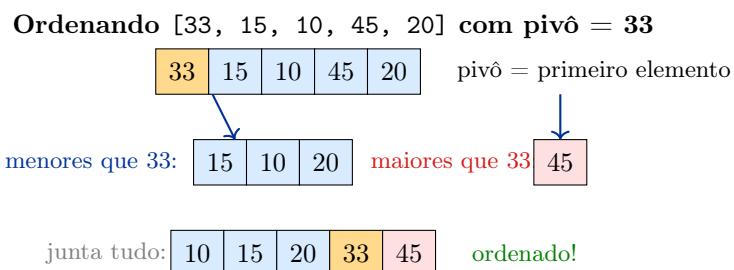
Esta é a última aula do nosso bloco de C e algoritmos. Reunimos tudo: **recursão** (Aulas 8-9), **dividir para conquistar** (Aula 9) e **Big O** (Aula 10). O Quicksort usa os três ao mesmo tempo.

1. A ideia do Quicksort

Já conhecemos o **Selection Sort** (Aula 7, exercício de ordenação por seleção). Ele funciona, mas é $O(n^2)$ — lento para listas grandes. O **Quicksort** resolve o mesmo problema de forma muito mais rápida, em média $O(n \log n)$.

A estratégia é dividir para conquistar:

1. Escolha um elemento qualquer da lista — chamado **pivô**.
2. Separe os outros em dois grupos: os **menores** que o pivô e os **maiores** que o pivô.
3. Ordene cada grupo **recursivamente** com o mesmo método.
4. Junte tudo: menores ordenados + pivô + maiores ordenados.



2. O caso-base

Como toda recursão, o Quicksort precisa de um caso-base. Aqui ele é muito simples:

Caso-base do Quicksort

Uma lista com **0 ou 1 elemento** já está ordenada por definição. Não há o que fazer — basta devolvê-la como está.

A cada chamada recursiva, os grupos “menores” e “maiores” são sempre menores que a lista original, então cedo ou tarde chegamos a grupos de tamanho 0 ou 1 — o caso-base.

3. Implementação em C

A versão em C usa índices `inicio` e `fim` para trabalhar sobre o mesmo vetor, sem criar cópias. O coração do algoritmo é a função `particionar`, que reorganiza o vetor em torno do pivô.

```

#include <stdio.h>

/*
 * Particiona v[inicio..fim] usando o ultimo elemento como pivo.
 * Coloca o pivo na posicao correta e retorna seu indice final.
 * Tudo a esquerda fica menor, tudo a direita fica maior.
 */
int particionar(int v[], int inicio, int fim) {
    int pivo = v[fim];          /* escolhe o ultimo como pivo */
    int i = inicio - 1;         /* fronteira dos menores      */

    for (int j = inicio; j < fim; j++) {
        if (v[j] <= pivo) {
            i++;
            /* troca v[i] com v[j] usando variavel temporaria */
            int temp = v[i];
            v[i] = v[j];
            v[j] = temp;
        }
    }
    /* coloca o pivo na posicao correta: troca v[i+1] com v[fim] */
    int temp = v[i + 1];
    v[i + 1] = v[fim];
    v[fim] = temp;

    return i + 1;              /* retorna a posicao do pivo */
}

/* Quicksort recursivo */
void quicksort(int v[], int inicio, int fim) {
    if (inicio < fim) {         /* caso recursivo */
        int p = particionar(v, inicio, fim); /* posiciona o pivo */
        quicksort(v, inicio, p - 1);         /* ordena os menores */
        quicksort(v, p + 1, fim);           /* ordena os maiores */
    }
    /* se inicio >= fim: 0 ou 1 elemento, caso-base, nao faz nada */
}

int main() {
    int v[] = {33, 15, 10, 45, 20, 8, 50};
    int n = 7;

    printf("Antes: ");
    for (int i = 0; i < n; i++) printf("%d ", v[i]);

    quicksort(v, 0, n - 1);

    printf("\nDepois: ");
    for (int i = 0; i < n; i++) printf("%d ", v[i]);
    printf("\n");
    return 0;
}

```

A troca com variável temporária

Para trocar dois elementos de posição, usamos o mesmo padrão de três passos do Selection Sort (Aula 7): guardamos um valor numa variável `temp`, copiamos o segundo por cima do primeiro, e por fim copiamos o `temp` para o segundo. É a troca clássica de duas variáveis — nada de novo.

4. Por que Quicksort é mais rápido que Selection Sort

	Selection Sort	Quicksort
Estratégia	busca o menor, repete	divide e conquista
Big O (médio)	$O(n^2)$	$O(n \log n)$
Para 1.000 itens	$\sim 1.000.000$ operações	~ 10.000 operações
Para 1.000.000 itens	$\sim 10^{12}$ operações	$\sim 20.000.000$ operações

Para listas grandes, a diferença é de horas para segundos. É por isso que o Quicksort (ou variações dele) é usado nas bibliotecas padrão da maioria das linguagens — inclusive na função `qsort` da própria linguagem C.

Curiosidade: o pior caso do Quicksort

Se a lista já estiver ordenada e o pivô for sempre o primeiro (ou último) elemento, o Quicksort vira $O(n^2)$ — igual ao Selection Sort. Por isso implementações reais escolhem o pivô de forma esperta (elemento do meio ou aleatório). No caso *médio*, que é o mais comum, ele é $O(n \log n)$.

5. Laboratório Prático

Exercício 1: Quicksort com rastreamento

Adicione ao código do Quicksort uma impressão do vetor após cada particionamento. Rode com [33, 15, 10, 45, 20, 8, 50] e observe como o vetor vai se organizando passo a passo.

Exercício 2: Contador de comparações

Adicione um contador global que conta quantas comparações `v[j] <= pivo` são feitas. Compare o total com um Selection Sort no mesmo vetor de 10 elementos.

Exercício 3: Ordenar struct

Adapte o Quicksort para ordenar um vetor de `struct Aluno` (nome e nota) pela nota, em ordem decrescente. Reaproveite o que você aprendeu sobre struct nas Aulas 6 e 7.

Exercício 4 (Desafio): Comparar os três

Crie um programa que gera um vetor com 1.000 números aleatórios (use `rand()`) e ordena com:

- Selection Sort
- Quicksort

Conte as comparações de cada um e exiba os totais lado a lado. A diferença deve ser dramática.

Resumo da aula

- **Quicksort:** escolhe um pivô, separa menores e maiores, ordena cada grupo recursivamente.
- **Caso-base:** lista de 0 ou 1 elemento já está ordenada.
- **Big O médio:** $O(n \log n)$ — muito melhor que o $O(n^2)$ do Selection Sort.
- Combina os três conceitos do módulo: recursão, dividir para conquistar e análise de eficiência.

Encerramento do módulo de C

Com esta aula fechamos o bloco de C e algoritmos. Você saiu de variáveis e tipos e chegou a algoritmos de ordenação eficientes — um caminho que muitos cursos levam um ano inteiro para percorrer.

Próximo módulo: JavaScript. Toda a lógica que você construiu em C — `if`, `for`, funções, vetores — funciona quase igual em JavaScript. A diferença é que o JS roda no navegador e é a linguagem que você usará para construir o Projeto Integrador no próximo ano. A sintaxe muda pouco; o que você aprendeu de lógica é o que realmente importa.

GABARITO – USO DO PROFESSOR

Exercício 1: Quicksort com rastreamento

```
/* Adicionar dentro de quicksort, apos particionar: */

void quicksort(int v[], int inicio, int fim, int n) {
    if (inicio < fim) {
        int p = particionar(v, inicio, fim);

        printf("Pivo posicionado em %d: ", p);
        for (int i = 0; i < n; i++) printf("%d ", v[i]);
        printf("\n");

        quicksort(v, inicio, p - 1, n);
        quicksort(v, p + 1, fim, n);
    }
}

/* No main, chamar: quicksort(v, 0, n-1, n); */
```

Mostra o vetor se reorganizando a cada particionamento.

Exercício 2: Contador de comparações

```
#include <stdio.h>
int comparacoes = 0;    /* contador global */

int particionar(int v[], int inicio, int fim) {
    int pivo = v[fim], i = inicio - 1;
    for (int j = inicio; j < fim; j++) {
        comparacoes++;           /* conta cada comparacao */
        if (v[j] <= pivo) {
            i++;
            int t = v[i]; v[i] = v[j]; v[j] = t;
        }
    }
    int t = v[i+1]; v[i+1] = v[fim]; v[fim] = t;
    return i + 1;
}

void quicksort(int v[], int inicio, int fim) {
    if (inicio < fim) {
        int p = particionar(v, inicio, fim);
        quicksort(v, inicio, p - 1);
        quicksort(v, p + 1, fim);
    }
}

int main() {
    int v[] = {5, 2, 9, 1, 7, 3, 8, 4, 6, 0};
    quicksort(v, 0, 9);
    printf("Comparacoes do Quicksort: %d\n", comparacoes);
    /* Selection Sort no mesmo vetor faz ~45 comparacoes (n*(n-1)/2) */
    return 0;
}
```

Exercício 3: Ordenar struct

```
#include <stdio.h>
#include <string.h>

typedef struct { char nome[50]; float nota; } Aluno;

int particionar(Aluno v[], int inicio, int fim) {
    float pivo = v[fim].nota;
    int i = inicio - 1;
    for (int j = inicio; j < fim; j++) {
        /* >= para ordem DECRESCENTE */
        if (v[j].nota >= pivo) {
            i++;
            Aluno t = v[i]; v[i] = v[j]; v[j] = t;
        }
    }
    Aluno t = v[i+1]; v[i+1] = v[fim]; v[fim] = t;
    return i + 1;
}

void quicksort(Aluno v[], int inicio, int fim) {
    if (inicio < fim) {
        int p = particionar(v, inicio, fim);
        quicksort(v, inicio, p - 1);
        quicksort(v, p + 1, fim);
    }
}

int main() {
    Aluno t[4] = {{ "Ana", 7.5 }, { "Bruno", 9.0 }, { "Carla", 6.0 }, { "Davi", 8.5 } };
    quicksort(t, 0, 3);
    printf("=== RANKING ===\n");
    for (int i = 0; i < 4; i++)
        printf("%d. %-10s %.1f\n", i+1, t[i].nome, t[i].nota);
    return 0;
}
```

Trocar >= por <= no particionar inverte para ordem crescente.

Exercício 4: Comparar os três

```
#include <stdio.h>
#include <stdlib.h>

long compQS = 0, compSS = 0;

/* --- Quicksort --- */
int particionar(int v[], int ini, int fim) {
    int pivo=v[fim], i=ini-1;
    for (int j=ini; j<fim; j++) { compQS++;
        if (v[j]<=pivo){ i++;
            int t=v[i]; v[i]=v[j]; v[j]=t; } }
    int t=v[i+1]; v[i+1]=v[fim]; v[fim]=t;
    return i+1;
}

void quicksort(int v[], int ini, int fim) {
    if (ini<fim){ int p=particionar(v,ini,fim);
        quicksort(v,ini,p-1); quicksort(v,p+1,fim); }
}

/* --- Selection Sort --- */
void selectionSort(int v[], int n) {
    for (int i=0; i<n-1; i++) {
        int min=i;
        for (int j=i+1; j<n; j++) { compSS++;
            if (v[j]<v[min]) min=j; }
        int t=v[i]; v[i]=v[min]; v[min]=t;
    }
}

int main() {
    int a[1000], b[1000];
    for (int i=0; i<1000; i++) { int r=rand()%10000; a[i]=r; b[i]=r; }

    quicksort(a, 0, 999);
    selectionSort(b, 1000);

    printf("Quicksort:      %ld comparacoes\n", compQS);
    printf("Selection Sort: %ld comparacoes\n", compSS);
    return 0;
}
```

Selection Sort faz ~499.500 comparacoes (fixo). Quicksort faz tipicamente ~10.000 a 12.000. A diferença de quase 50x ilustra na prática $O(n^2)$ vs $O(n \log n)$.